Kora - Naira Rail
Master Build Specification v1.0

Canonical implementation specification for Kora's first production product.
NGN ↔ USDC-on-Stellar infrastructure for fintechs, wallets, payroll products, remittance apps, Stellar applications and other businesses.

Product thesis. Every product that wants reliable Nigeria ↔ Stellar access should not have to rebuild bank collection, payouts, KYC/KYB, reconciliation, USDC settlement, webhooks and operational recovery. Kora v1 packages that rail as infrastructure.

Document status	Canonical v1 build spec
v1 product	Naira Rail
Core corridor	NGN ↔ USDC on Stellar
Primary customers	Fintechs, wallets, Stellar apps, payroll/remittance/payment products
Primary surfaces	REST API, TypeScript SDK, hosted on/off-ramp, business/developer console, docs
System of record	Kora internal double-entry ledger
Settlement network	Stellar
Runtime principle	Lean modular architecture; no microservice explosion
Out of v1	Pan-African routing, merchant acquiring network, consumer mobile app, MCP/agents, custom Soroban

This specification intentionally makes the first Kora release narrower than the long-term Kora Network. The roadmap at the end shows how Naira Rail evolves into African financial interoperability infrastructure without rebuilding the financial core.

Contents

1. Canonical v1 definition and boundaries
2. Users, customers and jobs-to-be-done
3. v1 product surfaces
4. End-to-end money flows
5. Regulatory and partnership operating model
6. Architecture principles
7. Runtime topology - four deployable concerns, not eleven services
8. Domain model and terminology
9. Money library and currency registry
10. Double-entry ledger and chart of accounts
11. Payment Intent and state machines
12. Idempotency, exactly-once instructions and outbox
13. NGN bank rail integration
14. Stellar and USDC rail integration
15. Anchor Platform / SEP compatibility
16. Quotes, FX, fees and limits
17. KYC, KYB, AML, sanctions and risk
18. Reconciliation and exception management
19. Provider connector SDK
20. Public REST API
21. Webhooks and event catalog
22. TypeScript SDK
23. Hosted on-ramp and off-ramp surfaces
24. Business / developer console
25. Operations console
26. Sandbox and provider simulators
27. Security, custody and key management
28. Data architecture and privacy
29. Observability and incident response
30. Testing, financial invariants and chaos engineering
31. Infrastructure, environments and CI/CD
32. Repository structure
33. Documentation set
34. Team skills and operating responsibilities
35. Launch plan and phased delivery
36. Production readiness checklist
37. Success metrics and grant evidence
38. Explicit non-goals
39. Kora roadmap after Naira Rail
40. First engineering tickets
41. Canonical decision log
42. Reference architecture sources

1. Canonical v1 definition and boundaries

Kora v1 is Naira Rail: a reliable, documented and reconcilable NGN ↔ USDC-on-Stellar on/off-ramp that third-party products integrate instead of building Nigeria fiat connectivity themselves.

The company-level vision remains African financial interoperability infrastructure. The v1 implementation does not pretend that the pan-African network already exists. It proves the most reusable primitive first: collect NGN from Nigerian financial rails and deliver USDC on Stellar, and accept USDC on Stellar and deliver NGN to a verified Nigerian bank account.

The flagship is the rail. A hosted consumer surface exists to complete flows and demonstrate the rail, but the public API, operational reliability and partner integration are the product.

v1 must ship four complete capabilities:

- NGN → USDC on Stellar: collect NGN, confirm and reconcile receipt, perform conversion/settlement through the licensed partner/anchor/liquidity arrangement, and deliver allowlisted USDC to the specified Stellar destination.
- USDC on Stellar → NGN: receive/confirm allowlisted USDC, satisfy compliance and quote requirements, execute NGN payout to a verified bank account, and reconcile both sides.
- Developer infrastructure: API, TypeScript SDK, webhooks, sandbox, test fixtures, OpenAPI reference and integration guides.
- Operational infrastructure: double-entry ledger, reconciliation, compliance controls, provider health, exception queue, audit log, limits, refunds/returns and support tooling.

v1 is not a generic crypto exchange, wallet, neobank, pan-African router, merchant acquiring network, DeFi app or agent-controlled money system.

2. Users, customers and jobs-to-be-done

Persona	Job	What Kora provides
Stellar wallet/app	Offer Nigeria fiat access	Programmatic deposit/withdraw or hosted ramp; status + webhooks
Fintech/remittance app	Move between NGN and Stellar USDC	API, quotes, KYC orchestration, payout/collection, reconciliation
Payroll/product platform	Cash out USDC to Nigerian beneficiaries	Beneficiary validation, payout API, limits, events
Treasury/business	Acquire or redeem USDC	Business dashboard, balances, quotes, transaction history
Kora operations	Resolve real-money exceptions	Attempts, journals, provider evidence, recon queue, freezes
Compliance team/partner	Control who/what can move	KYB/KYC status, screening, holds, immutable decisions

3. v1 product surfaces

Ship only surfaces that directly support the rail:

Surface	Required in v1	Purpose
Public API	Yes	Canonical programmable interface for integrators
TypeScript SDK	Yes	Typed API client + webhook verification + idempotency helpers
Hosted Ramp	Yes	Mobile-first hosted NGN→USDC and USDC→NGN flow

Surface	Required in v1	Purpose
Business/Developer Console	Yes	Keys, webhooks, logs, balances, transactions, limits
Operations Console	Yes	Recon, exceptions, provider health, compliance holds, journal visibility
Docs	Yes	Quickstarts, API reference, SEPs, sandbox, errors, webhooks
Python SDK	Roadmap	Add when design partners require it
Consumer mobile app	Roadmap	Not needed to prove the rail
MCP/agent	Roadmap	Only after authorization and reconciliation are mature

4. End-to-end money flows

4.1 NGN → USDC on Stellar

```
Client creates on-ramp intent
  -> Kora authenticates + idempotency check
  -> compliance/limits pre-check
  -> quote is created/locked
  -> Kora returns NGN funding instructions
  -> payer sends NGN via partner bank/PSP
  -> provider receipt arrives
  -> worker reconciles expected amount/reference
  -> ledger posts NGN collection
  -> conversion/settlement instruction is created
  -> USDC delivery is submitted to Stellar
  -> Stellar transaction is confirmed
  -> chain evidence is matched to expected asset/issuer/amount/destination
  -> ledger posts settlement
  -> intent = succeeded
  -> webhook delivered
```

4.2 USDC on Stellar → NGN

```
Client creates off-ramp intent
  -> beneficiary bank account resolved/verified
  -> compliance/limits pre-check
  -> quote is created/locked
  -> Kora returns Stellar USDC deposit instructions
  -> Kora observes expected USDC
  -> verify network + asset code + issuer + amount + reference
  -> ledger posts digital-asset receipt
  -> NGN liquidity/partner instruction reserved
  -> exactly-once NGN payout instruction sent
  -> bank/PSP acknowledgement recorded
  -> final provider evidence reconciled
  -> intent = succeeded OR returned/failed/unknown
  -> webhook delivered
```

4.3 Failure principle

No ambiguous provider timeout may be interpreted as failure and blindly retried. If Kora cannot know whether a money-moving instruction landed, the attempt becomes unknown and enters status/reconciliation recovery.

5. Regulatory and partnership operating model

Licensing/regulated fiat handling is part of the product architecture, not a launch footnote. Kora v1 must have a documented operating model agreed with counsel and the relevant regulated partner(s) before production money.

Required workstreams:

- Determine Kora's legal role in the launch model: technology provider, agent, sub-merchant, PSP partner, principal/limited principal or other counsel-approved structure.

- Obtain a signed commercial/technical partnership path with a licensed Nigerian bank/PSP/regulated fiat handler for NGN collection and payout.
- Define which party performs KYC/KYB, AML/sanctions screening, transaction monitoring, suspicious-activity escalation, customer support and regulatory reporting.
- Define custody and safeguarding: who legally holds NGN float, who holds USDC, whether Kora ever takes custody, and what customer funds protections apply.
- Define settlement, reserve, prefunding, reversal, chargeback/return, refund, downtime and reconciliation SLAs in partner agreements.
- Maintain country/corridor limits and compliance requirements as configuration, not hard-coded assumptions.

Production launch gate: no live money until legal role, partner responsibilities, custody, data processing, compliance ownership and incident escalation are documented and approved.

6. Architecture principles

- Internal double-entry ledger is canonical for Kora balances and obligations.
- Stellar is the settlement/digital-asset rail, not the application database.
- Provider callbacks and Stellar transactions are evidence that must be matched and journaled.
- One public Payment Intent model hides provider-specific implementation.
- Exactly-once external money instructions; at-least-once internal processing.
- Compliance and limits sit on the write path before money moves.
- Reconciliation is continuous + scheduled and is mandatory.
- OpenAPI is the public API contract; SDK types are generated from it.
- Start modular; split a module only when a named SLA, team ownership or throughput constraint requires it.
- No custom Soroban contract unless an existing Stellar/anchor primitive cannot safely satisfy the requirement.

7. Runtime topology - four deployable concerns, not eleven services

```
apps/
  business-console
  hosted-ramp
  operations
  docs

API Gateway
  auth | idempotency | rate limit | request IDs
        |
        +------------------------+
        |                        |
payments-api                platform-api
intents                     merchants
quotes                      API keys
balances                    webhook endpoints
customers/instruments       org settings
asset registry
routing policy (simple)
treasury views
        |
payments-worker
outbox | attempts | provider I/O | Stellar execution
```

```
webhook fan-out | recon jobs | notification jobs
        |
  ledger module
  compliance module
  connector modules

connectors:
  ngn-bank/
  stellar-anchor/
  stellar-network/
  fx-liquidity/
  fake/
```

The ledger and compliance modules may run in-process initially. The worker is the only ordinary writer of financial journals after API approval/hold creation. A sealed operations path may perform exceptional journals with two-person approval.

8. Domain model and terminology

Object	Meaning
Merchant	API customer / integrating organization
Customer	End person/business associated with a merchant flow
Instrument	Bank account or Stellar account used to send/receive value
Rail	NGN bank rail or Stellar USDC rail
Intent	Canonical user instruction: on_ramp, off_ramp, refund
Attempt	One provider/network execution attempt for an intent
Instruction	Exactly-once outbound order to a provider/network signer
Receipt	Inbound evidence: provider callback/statement or Stellar tx
Quote	Time-bounded NGN/USDC exchange and fee commitment
Journal/Entry	Immutable double-entry accounting record
Account	Ledger account in one currency/asset
Reconciliation Item	Mismatch, missing evidence or unmatched external movement
Provider	Licensed partner, anchor, custody partner, FX/LP, Stellar RPC provider
Asset	NGN or uniquely identified Stellar USDC (network + code + issuer)

9. Money library and currency registry

Build packages/money first. Never use JavaScript floating-point numbers for money.

```
type AssetId =
  | { kind: 'fiat'; code: 'NGN'; scale: 2 }
  | { kind: 'stellar'; network: 'testnet'|'public';
      code: 'USDC'; issuer: string; scale: 6 };

class Money {
  asset: AssetId;
  minor: bigint;
  add(other): Money;      // same asset only
  sub(other): Money;
  cmp(other): -1|0|1;
  toJSON(): { asset: ..., minor: string };
}

class Quote {
  id: string;
  source: Money;
```

```
  destination: Money;
  rate: string;           // decimal string
  fee: Money;
  expiresAt: Date;
  providerRef?: string;
}
```

- All database amounts are integer minor units / stroops-equivalent scale as appropriate.
- Scale belongs to the asset registry; do not hard-code 2 decimals.
- USDC identity must include issuer + network; never trust ticker alone.
- JSON serializes integer amounts as strings to avoid precision loss.
- FX conversion is explicit through a Quote/FxTicket; Money never performs implicit conversion.
- Rounding mode must be explicit and tested.

10. Double-entry ledger and chart of accounts

Every external money movement has a corresponding internal journal. Entries are append-only; corrections use reversing journals, never edits.

Account purpose	Example
NGN safeguard / partner cash	sys:safeguard:NGN
USDC treasury/custody	sys:treasury:USDC:
NGN provider clearing	sys:provider_clearing:ngn_bank
USDC Stellar clearing	sys:provider_clearing:stellar_usdc
Fees	sys:fees:NGN / sys:fees:USDC
FX wash	sys:fx_wash:
Merchant pending	mcht::pending:
Merchant available	mcht::available:
Merchant reserve	mcht::reserve:

Ledger invariants:

- Per journal and per asset: total debits equal total credits.
- No entry without a journal; no UPDATE/DELETE on posted entries.
- Ledger account asset is immutable.
- Available funds are held/reserved before any outbound payout/settlement instruction.
- Only the ledger computes Kora balances; external provider balances are reconciliation inputs.
- Concurrency uses database locking/serializable strategy so two payouts cannot spend the same available balance.
- Every fee is explicit and journaled; never silently alter principal.

11. Payment Intent and state machines

Use one public object for both directions.

```
Intent {
  id
  type: on_ramp | off_ramp | refund
  merchant_id
  source_amount
```

```
  destination_amount?
  quote_id?
  customer_id?
  source_instrument?
  destination_instrument?
  state
  compliance_state
  metadata
  created_at
  updated_at
}
```

On-ramp states

```
created -> requires_action -> funding_pending -> funds_received -> settling -> succeeded
                         \-> expired/canceled
funds_received/settling -> failed | refund_pending -> refunded
```

Off-ramp states

```
created -> requires_action -> asset_pending -> asset_received -> payout_pending -> processing -> succeeded
                                                                              -> failed | unknown | returned
```

A returned Nigerian payout after an apparent success is a first-class transition. Do not model it as an impossible edge case.

12. Idempotency, exactly-once instructions and outbox

- Idempotency-Key is mandatory on every public mutating endpoint.
- Store key + merchant + endpoint + SHA-256 request fingerprint + original response.
- Same key + same payload returns original resource; same key + different payload returns 409.
- Create provider/network Instruction row before the external call; enforce a uniqueness key that prevents a second logical send.
- Worker delivery is at-least-once, but external instruction creation is exactly-once.
- Write state change + outbox event in the same Postgres transaction.
- Never blindly retry an ambiguous POST to a bank/PSP/anchor. Query status or reconcile first.

13. NGN bank rail integration

Kora must integrate at least one real NGN collection method and one real NGN payout method through a licensed partner. The connector boundary prevents partner-specific semantics from leaking into the public API.

Required NGN capabilities:

- Pay-with-transfer / virtual account or partner-supported bank collection.
- Bank account resolution/name enquiry before payout.
- Payout initiation with merchant reference and Kora intent reference.
- Provider status query.
- Signed/authenticated provider webhook parsing.
- Statement/export retrieval for reconciliation if available.
- Explicit provider limits, operating hours and known failure codes.
- Return/reversal handling.

- Sandbox/test environment or Kora-recorded fixtures.

14. Stellar and USDC rail integration

Stellar is the digital settlement side of the rail. Use official Stellar SDKs and current network APIs. Kora must ingest the network events it needs into its own data store; do not rely on RPC as historical storage.

Required Stellar capabilities:

- Testnet and Public Network configuration with separate keys and endpoints.
- Allowlisted USDC definition by network + asset code + issuer.
- Destination account validation and trustline validation where the destination must hold classic USDC.
- Transaction construction/signing/submission through a hardened signing path.
- Transaction status confirmation and durable indexing into Kora's receipts table.
- Memo/reference strategy that maps network movement to Kora Intent/Attempt without depending on memo alone.
- Sequence-number concurrency handling.
- Fee handling and timeout/retry rules.
- Wrong issuer, wrong network, wrong destination, wrong amount and duplicate transaction protection.
- RPC provider health/failover strategy for production.

15. Anchor Platform / SEP compatibility

Kora v1 should be usable by Stellar wallets/apps as an ecosystem-compatible ramp. Anchor Platform currently supports SEP-1, SEP-6, SEP-10, SEP-12, SEP-24, SEP-31, SEP-38 and SEP-45. For the Naira Rail, prioritize the SEPs that directly improve discovery, authentication, KYC, quotes and deposit/withdraw interoperability.

SEP	v1 treatment
SEP-1	Required for public anchor discovery: stellar.toml, organization, assets, endpoints
SEP-6	Target programmatic integration for wallets/apps where Kora/partner model supports it
SEP-10	Support account authentication where required by SEP flows
SEP-12	Use where KYC data exchange is part of the chosen SEP-6 integration model
SEP-24	Hosted alternative for integrations where Kora/partner owns interactive KYC/flow
SEP-38	Use for standardized quote/exchange information where compatible with operating model
SEP-31	Not required to prove v1 Naira Rail; reserve for later cross-border receive-side flows
SEP-45	Add when supporting contract-account authentication is a real client requirement

16. Quotes, FX, fees and limits

- Quote must expose source amount, destination amount, rate, Kora fee, provider/partner fee if disclosed, expiry and total effective result.
- Quotes are short-lived and immutable after acceptance.
- Worker re-validates quote validity before external execution.
- Limits can be configured by merchant, customer tier, rail, direction, day/month and partner.
- Never promise a rate beyond the liquidity/partner commitment window.

- v1 does not require automated hedging; use partner/LP quote + configured Kora spread/fee.
- Store quote source and reconciliation evidence for every completed conversion.

17. KYC, KYB, AML, sanctions and risk

The exact regulated workflow depends on the agreed partner/legal model. The software must support either Kora-led or partner-led verification without assuming one universal KYC process.

- Merchant KYB before live keys.
- Customer identity profile with verification status and provider-specific requirement state.
- Sanctions/PEP screening through a licensed/credible vendor selected with counsel/compliance.
- Transaction velocity rules and amount thresholds.
- New-beneficiary and unusual-behavior holds.
- Source-of-funds/source-of-wealth escalation fields where required.
- Immutable compliance decision log: allow, hold, deny, reviewer, rule/version, evidence reference.
- Freeze capability for merchant/customer/instrument/intents.
- Data minimization: do not copy partner KYC documents unless Kora must legally/operationally store them.

18. Reconciliation and exception management

Reconciliation is a core product feature. A ramp that cannot prove what happened to money is not production infrastructure.

- Continuously match provider webhooks and Stellar receipts to expected Attempts/Instructions.
- Nightly fetch/import provider statements and compare against internal clearing accounts.
- Match by provider reference first; then controlled amount/time/account/reference heuristics.
- Unmatched external money creates a reconciliation item; it never silently changes a balance.
- Missing expected provider/network evidence creates an exception and alert.
- Returned payouts create reversing journals and a first-class Intent transition.
- Manual financial correction requires reason, evidence, actor and two-person approval.
- Daily reconciliation report must show opening balance, movements, closing balance, unmatched items and aged exceptions.

19. Provider connector SDK

```
interface Connector {
  id: string;
  capabilities: Array<
    'collect' | 'payout' | 'resolve' | 'refund' |
    'quote' | 'status' | 'statement'
  >;

  resolveInstrument?(req): Promise<ResolveResult>;
  getQuote?(req): Promise<ProviderQuote>;
  collect(req): Promise<ProviderAck>;
  payout(req): Promise<ProviderAck>;
  refund?(req): Promise<ProviderAck>;
  fetchStatus(providerRef): Promise<ProviderStatus>;
  parseWebhook(headers, rawBody): Promise<ProviderEvent>;
  fetchStatement?(from, to): Promise<StatementLine[]>;
}
```

- ProviderAck means accepted/rejected/pending, never 'money moved' unless final evidence supports that claim.
- Explicit connect/total timeouts on every provider call.
- Closed error taxonomy: invalid_account, insufficient_float, limit_exceeded, duplicate, timeout, unavailable, rejected, unknown.
- Timeout after a potentially accepted money instruction -> attempt unknown; do not blind retry.
- Every provider call gets correlation ID, trace, redacted request/response record and latency metric.
- Fake connector implements the same interface for sandbox/E2E/chaos testing.

20. Public REST API

Base: /v1. Server authentication uses secret API keys. OpenAPI is source of truth.

Core endpoints

```
GET    /v1/me
GET    /v1/balances
GET    /v1/assets
GET    /v1/limits

POST   /v1/customers
GET    /v1/customers/:id
POST   /v1/customers/:id/instruments
GET    /v1/customers/:id/instruments

POST   /v1/instruments/resolve

POST   /v1/quotes
GET    /v1/quotes/:id

POST   /v1/intents
GET    /v1/intents/:id
GET    /v1/intents
POST   /v1/intents/:id/cancel
POST   /v1/intents/:id/refunds

POST   /v1/webhook_endpoints
GET    /v1/webhook_endpoints
POST   /v1/webhook_endpoints/:id/rotate_secret
POST   /v1/webhook_endpoints/:id/test

GET    /v1/events
POST   /v1/events/:id/replay
```

Example on-ramp intent

```
POST /v1/intents
Idempotency-Key: order-9182

{
  "type": "on_ramp",
  "source": {
    "asset": "NGN",
    "amount_minor": "15000000"
  },
  "destination": {
    "asset": {
      "network": "stellar",
      "code": "USDC",
      "issuer": "ALLOWLISTED_ISSUER"
    },
    "stellar_address": "G..."
  },
  "customer_id": "cus_..."
}
```

Example off-ramp intent

```
POST /v1/intents
Idempotency-Key: cashout-778

{
  "type": "off_ramp",
  "source": {
    "asset": {
      "network": "stellar",
      "code": "USDC",
      "issuer": "ALLOWLISTED_ISSUER"
    },
    "amount_minor": "100000000"
  },
  "destination": {
    "type": "bank_account",
    "bank_code": "...",
    "account_number": "...",
    "account_name": "..."
  },
  "customer_id": "cus_..."
}
```

API rules

- Cursor pagination; stable sort order.
- Request/response IDs and correlation IDs.
- 409 for idempotency payload mismatch; 422 for business-rule rejection; 429 for rate/volume limit; 503 for temporarily unavailable rail.
- Provider names do not leak unless contractually/usefully exposed; public API speaks Kora rails and instruments.
- Sensitive bank data is masked in responses and logs.
- Every error code has a documentation anchor and machine-readable type/code.

21. Webhooks and event catalog

Webhook delivery is at-least-once. Merchants deduplicate by event ID. Sign raw body using HMAC-SHA256 with timestamped signature header and reject stale timestamps in SDK helper.

```
intent.created
intent.requires_action
intent.funding_pending
intent.funds_received
intent.asset_received
intent.processing
intent.succeeded
intent.failed
intent.unknown
intent.returned
intent.canceled
intent.expired
refund.succeeded
balance.updated
compliance.action_required
```

Retry schedule should use exponential backoff and expose failed deliveries/replay in the console.

22. TypeScript SDK

Package: @kora/sdk. Generate models from OpenAPI; hand-write only ergonomic helpers.

- Typed Kora client.
- Automatic request IDs and configurable timeout.

- Idempotency-key option on every mutating method.
- No automatic unsafe POST retry without the same idempotency key.
- Pagination helpers.
- Webhook signature verification helper.
- Money/asset types re-exported.
- Clear KoraError classes with HTTP status, type, code and request ID.
- Test/live key validation.

23. Hosted on-ramp and off-ramp surfaces

Hosted Ramp is a thin first-party client of the same public-domain APIs. It is not a second payment stack.

- Mobile-first.
- Merchant branding within strict safe bounds.
- Amount, quote, fees, destination/beneficiary summary.
- NGN transfer instructions and copy affordance for on-ramp.
- Stellar deposit instructions for off-ramp.
- KYC/verification step only when required.
- Live status page through terminal state.
- Clear expiry and recovery states.
- No crypto jargon beyond what is necessary to identify USDC/Stellar destination.
- No wallet portfolio, trading UI, token discovery or speculative features.

24. Business / developer console

- Home: balances, live-mode status, recent intents, failures, limits.
- Transactions: filters by direction/state/customer/date/reference.
- Customers and instruments.
- Developers: test/live API keys, webhook endpoints, event log, request inspector, SDK/docs links.
- Settlement/reconciliation summary appropriate to merchant visibility.
- Organization settings, team roles, security.
- KYB/live activation status.

25. Operations console

- Intent debugger: full state history, attempts, instruction IDs, receipts, journals.
- Provider health and latency.
- Reconciliation queue with aging.
- Unknown/ambiguous attempts.
- Compliance holds and review cases.
- Merchant/customer/instrument freeze.
- Manual journal/reversal tool behind two-person approval.
- Raw provider payloads redacted by default with privileged reveal.

- Stellar receipt/transaction evidence.
- Liquidity/float view for NGN and USDC partner accounts.
- Audit log.

26. Sandbox and provider simulators

The sandbox is part of the product. A design partner should integrate before Kora gives them production access.

- Test API keys and isolated data.
- Fake NGN bank connector.
- Stellar Testnet path.
- Deterministic test customer/instrument values.
- Scenarios: success, insufficient float, invalid bank account, quote expiry, compliance hold, provider timeout, duplicate callback, payout returned, Stellar wrong issuer, Stellar missing trustline, webhook failure.
- Request inspector and event replay.
- Recorded provider fixtures for connector contract tests.
- Sandbox must never share production secrets, signing keys, balances or webhook endpoints.

27. Security, custody and key management

- TLS everywhere; HSTS on public web surfaces.
- Secrets in managed secret store, never committed or stored in plaintext env files in production.
- Production Stellar signing through HSM/MPC/KMS/custody architecture appropriate to the operating model; no raw seed in application environment.
- Separate hot operational limits from treasury/cold controls.
- API keys stored hashed; show secret once; prefix identifies test/live.
- WebAuthn/passkey or strong MFA for operations and privileged business users.
- Least privilege IAM; production database access time-bound and audited.
- PII field encryption for account numbers/identity values.
- WAF/rate limits; per-key and per-merchant volume controls.
- Dependency/SAST/secret scanning in CI.
- Independent penetration test before live money.
- Incident key rotation and emergency payout/settlement stop controls.

28. Data architecture and privacy

Store	v1 use
PostgreSQL	Merchants, customers, instruments, intents, attempts, instructions, quotes, ledger, outbox, receipts, recon, audit
Redis	Rate limits, short-lived locks/reservations, sessions, cache; never authoritative balances
Object storage	KYC/statement evidence only where required; encrypted and access-controlled
Analytics warehouse	Not required for v1; add when operational/reporting volume warrants it

- UTC internally.
- UUIDv7/ULID with typed prefixes.
- Retention schedule per data class.
- Data minimization and deletion/anonymization process subject to financial/legal retention obligations.
- Backups with point-in-time recovery and tested restores.
- Stellar receipts needed for Kora history are persisted internally because RPC is not a historical index.

29. Observability and incident response

- OpenTelemetry traces from API request through worker/provider/Stellar execution.
- Structured JSON logs with intent_id, attempt_id, instruction_id, merchant_id, provider, request_id.
- Metrics: intent state counts, success/failure, provider latency/error rate, ledger append latency, webhook delivery, recon age, unknown attempts, float levels, Stellar submission/confirmation.
- Alerts: ledger invariant failure; duplicate-instruction constraint attempt; unknown payout; recon aging; provider outage; low float; webhook failure spike; signer/RPC failure.
- Runbooks for NGN provider outage, Stellar/RPC outage, low liquidity, signing failure, webhook backlog, reconciliation mismatch, compromised API key and partner incident.

30. Testing, financial invariants and chaos engineering

Layer	Required tests
Money	Property tests for arithmetic, scale, serialization, rounding
Ledger	Balanced journals, immutable entries, concurrent holds, rollback on failure
State machine	Allowed/forbidden transitions and terminal-state behavior
Connector contract	Recorded fixtures, timeout mapping, duplicate callbacks, error normalization
Integration	API + worker + Postgres + Redis + fake connectors
Stellar	Testnet send/receive, wrong issuer, trustline failure, duplicate receipt
E2E	Hosted ramp/API through fake NGN + Stellar Testnet + webhook receiver
Chaos	Kill worker after external instruction but before ack; prove no double send
Load	Ledger contention, idempotency, webhook fan-out, quote traffic
Security	Auth, authorization, secret scanning, webhook forgery, rate limits, PII redaction

Required invariant: same logical payout/settlement instruction across N worker crashes produces zero or one external instruction - never two.

31. Infrastructure, environments and CI/CD

- Local: Docker Compose with Postgres, Redis, API, worker, fake connector and local webhook receiver.
- Environments: local -> shared dev -> staging with real sandboxes/Testnet -> production.
- Containers built from immutable Git SHA.
- API and worker independently deployable.
- Start one region; no Kubernetes requirement for a small team.

- Terraform/OpenTofu modules for network/database/cache/object store/secrets/CI identity; apply only as operational needs justify.
- Expand/contract database migrations; never destructive in-place ledger migration.
- GitHub Actions: lint, typecheck, unit, ledger invariant, integration, security scan, build, migration check, staging E2E, gated production deploy.
- Production rollback strategy must not roll back already-posted money state; application rollback and data migrations are planned separately.

32. Repository structure

```
kora/
■■■ apps/
■   ■■■ business/          # merchant + developer console
■   ■■■ ramp/              # hosted on/off-ramp
■   ■■■ operations/        # internal ops
■   ■■■ docs/              # docs + OpenAPI explorer
■
■■■ services/
■   ■■■ api/               # platform + payment HTTP
■   ■■■ worker/            # outbox, providers, Stellar, webhooks, recon
■
■■■ packages/
■   ■■■ money/
■   ■■■ ledger/            # in-process first
■   ■■■ compliance/        # in-process first
■   ■■■ database/
■   ■■■ events/
■   ■■■ auth/
■   ■■■ provider-sdk/
■   ■■■ stellar/
■   ■■■ shared/
■   ■■■ observability/
■
■■■ connectors/
■   ■■■ ngn-bank/
■   ■■■ stellar-anchor/
■   ■■■ stellar-network/
■   ■■■ fx-liquidity/
■   ■■■ fake/
■
■■■ specs/
■   ■■■ openapi/kora.yaml
■   ■■■ events/catalog.yaml
■   ■■■ ledger/invariants.md
■   ■■■ providers/contract.md
■   ■■■ stellar/usdc.md
■   ■■■ compliance/decision-model.md
■
■■■ sdks/
■   ■■■ typescript/
■
■■■ infrastructure/
■   ■■■ docker/
■   ■■■ terraform/
■
■■■ tests/
    ■■■ integration/
    ■■■ e2e/
    ■■■ providers/
    ■■■ ledger/
    ■■■ stellar/
    ■■■ chaos/
```

Do not scaffold in v1: eleven domain microservices, Kubernetes, MCP, skills, mobile app, Python SDK, custom Soroban contracts, multi-country routing, Kafka, automatic netting or automated FX hedging.

33. Documentation set

- Overview: what Naira Rail is and who integrates it.
- Quickstart: sandbox NGN→USDC and USDC→NGN.
- Authentication and API keys.
- Idempotency.
- Customers and instruments.
- Quotes and fees.
- On-ramp lifecycle.
- Off-ramp lifecycle.
- Hosted Ramp.
- Webhooks with TypeScript verification example.
- Errors and recovery.
- Limits and compliance states.
- Assets: exact Stellar USDC identity and network rules.
- Stellar integration: account/trustline/reference expectations.
- SEP compatibility and stellar.toml.
- Sandbox test values and failure scenarios.
- Reconciliation/status semantics: succeeded vs processing vs unknown vs returned.
- TypeScript SDK.
- OpenAPI reference.
- Changelog and versioning policy.
- Security overview and responsible disclosure.
- Status page.

34. Team skills and operating responsibilities

Role/skill	Responsibility
Tech lead / payments backend	Ledger invariants, state machines, idempotency, architecture
Backend engineer(s)	API, worker, connector integration, webhooks, recon
Stellar engineer	SDK/RPC, USDC asset handling, anchor/SEP integration, signing, Testnet/Mainnet
Full-stack engineer	Business console, hosted ramp, operations UI, docs
Security/platform	Secrets, IAM, custody/signing integration, CI/CD, observability
Compliance/legal partner	Operating model, KYC/KYB/AML, privacy, partner responsibilities
Payments operations	Bank/PSP relationships, settlement, float, recon, incident handling
Partnerships/BD	Design partners, wallet/fintech integrations, production pilots

A small team may combine roles, but the responsibilities cannot be omitted.

35. Launch plan and phased delivery

Phase	Build	Exit criterion
0 - Financial core	Monorepo, CI, money library, Postgres schemas, ledger, fake connector	Property + ledger invariant tests green
1 - Payment engine	Intent API, idempotency, worker, outbox, fake collect/payout, webhooks	E2E fake flow survives retries/crashes
2 - NGN rail	Real sandbox collect/payout, name enquiry, statements, compliance stub, recon	10+ sandbox flows; forced worker restarts produce 0 duplicates
3 - Stellar rail	USDC allowlist, Testnet receive/send, chain receipts, trustline checks, signer abstraction	Closed Testnet USDC flow fully journaled/reconciled
4 - Naira Rail closed loop	NGN→USDC + USDC→NGN with quotes, partner liquidity, hosted ramp	Both directions complete end-to-end in staging
5 - Developer product	TS SDK, sandbox scenarios, docs, console, webhook replay, status semantics	External design partner integrates without engineer handholding
6 - Production hardening	KYB/KYC, real partner controls, pen test, ops/recon, alerting, limits, backups	Legal/partner/security launch gates signed off
7 - Mainnet pilot	Capped production traffic, live USDC, live NGN, daily recon	First real integrations and clean settlement evidence
8 - Grant/scale evidence	More integrators, volume, reliability, partner SLA data	Metrics support expansion decision

36. Production readiness checklist

Do not launch because the UI looks complete. Launch only when every applicable item is true.

- Signed/approved regulated operating model and partner responsibilities.
- Live NGN collection and payout credentials tested.
- USDC issuer/network allowlist verified for Public Network.
- Production signer/custody path approved; no raw production seed in app env.
- Ledger invariants and nightly invariant job green.
- Idempotency and exactly-once instruction chaos tests green.
- Continuous + statement reconciliation working.
- Unknown/returned payout handling tested.
- KYC/KYB/sanctions/limits production vendors/configuration enabled as required.
- Merchant live activation gated by KYB/contract/limits.
- Webhooks signed, retried and replayable.
- Backups and restore test completed.
- Operational alerts and runbooks tested.
- Pen test findings remediated or formally accepted.
- Provider and Stellar outage behavior tested.
- Support/escalation ownership defined.
- Daily settlement/reconciliation report reviewed during pilot.
- Volume caps enabled for pilot.

37. Success metrics and grant evidence

Measure infrastructure quality, not vanity signups.

Metric	Target / interpretation
Duplicate external payout/settlement instructions	0
Ledger invariant failures in production	0
Terminal intents matched to external evidence	≥99.9% within defined reconciliation window
Webhook eventual delivery	≥99.5% excluding invalid merchant endpoints
Time to first sandbox transaction	<30 minutes for a competent developer
Unknown attempts	Tracked and aged; no silent retries
NGN→USDC success rate	Tracked by provider, amount band and failure cause
USDC→NGN success rate	Tracked by provider, amount band and failure cause
P50/P95 completion time	Tracked separately per direction
Reconciliation exceptions	Aged, owned, and driven toward zero
External integrations	Named design partners using sandbox/live API
Stellar activity	Provable mainnet USDC settlement volume/transactions when live

Grant-ready evidence should include: licensed-partner/PSP relationship or letter, working sandbox, OpenAPI + SDK, live/Testnet demonstration, reconciliation evidence, external integrator(s), security/compliance plan, and milestones tied to real Stellar usage rather than UI completion.

38. Explicit non-goals

- No pan-African country matrix in v1.
- No KES/GHS/ZAR/XOF rails in v1.
- No cross-border fiat-to-fiat routing in v1.
- No general merchant card acquiring/Stripe-for-Africa product in v1.
- No consumer wallet or dollar-bank product.
- No token marketplace/trading/DEX.
- No arbitrary Stellar assets; USDC only unless a later approved asset is explicitly added.
- No custom stablecoin issuance by Kora.
- No autonomous agent with unrestricted money movement.
- No MCP server in v1.
- No mobile app requirement.
- No Kafka/event-bus cluster.
- No Kubernetes requirement.
- No eleven microservices.
- No custom Soroban contract solely for grant optics.
- No automatic netting, rebalancing or hedging.

39. Kora roadmap after Naira Rail

The roadmap expands the same ledger, Intent, connector and reconciliation core. Do not rebuild the foundation.

Roadmap stage	Capability added	What stays unchanged
R1 - More NGN integrations	Second NGN provider, failover, better liquidity	Ledger, API model, connector contract
R2 - Kora Payments	Hosted checkout/payment links/merchant settlement if demand proves it	Intent/ledger/webhooks; new payment capabilities as modules
R3 - Second African rail	KES or GHS mobile money/bank rail	Connector model + ledger
R4 - Cross-border	NGN→Stellar→KES/GHS and reverse	RoutePlan added; same intents/journals/recon
R5 - Kora Router	Multi-provider graph, health/liquidity-aware route choice	Routing remains module until scale forces split
R6 - Asset access	Additional allowlisted Stellar assets where legally/product justified	Asset registry expands
R7 - Institutional network	Banks/fintechs/anchors/LPs as network participants	Same API/connector primitives
R8 - Consumer/mobile	First-party Kora app as a client of the same APIs	No privileged payment stack
R9 - MCP/agents	Read tools then constrained money tools with policies/approvals	Same API authorization + ledger
R10 - Advanced treasury	Netting/rebalancing/liquidity optimization when volume warrants it	Ledger remains canonical

Long-term definition:

Kora becomes African financial interoperability infrastructure connecting financial institutions, fintechs, banks, mobile-money networks, anchors and liquidity providers through Stellar, including on-ramp, off-ramp, cross-border settlement and Stellar asset purchasing.

Naira Rail is the first production rail of that network, not a disposable MVP.

40. First engineering tickets

Implement in this order; do not parallelize the foundation into decorative surfaces.

#	Ticket
1	Scaffold pnpm/Turborepo monorepo + Docker Compose Postgres/Redis.
2	Implement packages/money with bigint, asset identity, serialization and property tests.
3	Create Drizzle schemas: merchants, customers, instruments, intents, attempts, instructions, quotes, ledger, outbox, receipts, recon.
4	Implement ledger postJournal + invariants + concurrent hold tests.
5	Implement API auth + Idempotency-Key middleware + request IDs.
6	Implement POST/GET /v1/intents with state transition guard.
7	Implement worker + transactional outbox.
8	Implement fake connector and fake NGN collect/payout.
9	Implement signed webhook delivery + TS verification helper.
10	Chaos test: kill worker after instruction creation / before acknowledgement; prove no duplicate.
11	Build minimal operations intent debugger + journal view.
12	Implement real NGN sandbox connector: collect, payout, resolve, status, webhook, statement.
13	Implement reconciliation receipts + recon queue.

#	Ticket
14	Implement compliance decision interface + merchant live gating.
15	Implement Stellar asset registry and exact USDC allowlist.
16	Implement Stellar Testnet connector: receive/send/status/receipt ingestion/trustline checks.
17	Implement quote model + FX/partner connector.
18	Implement closed NGN→USDC staging flow.
19	Implement closed USDC→NGN staging flow.
20	Build hosted ramp against public API.
21	Publish OpenAPI + docs quickstart + sandbox test scenarios.
22	Publish @kora/sdk TypeScript package.
23	Build business/developer console for keys/webhooks/logs/transactions.
24	Complete security/custody integration and production signer path.
25	Complete KYB/KYC/AML production integration and partner launch checklist.
26	Run pen test, backup restore, outage drills and production pilot readiness review.

41. Canonical decision log

Decision	Canonical v1 choice	Revisit when
Product	Naira Rail: NGN ↔ USDC on Stellar	After production reliability + integrator demand
System of record	Internal double-entry ledger	Never unless Kora becomes fully on-chain, which is not planned
Stellar role	USDC settlement/digital-asset rail	Expands with Kora Network
USDC identity	Network + code + issuer allowlist	Whenever approved issuer/network changes
Runtime	API + worker + web apps; ledger/compliance modules	Named SLA/team/throughput forces split
Bus	Postgres transactional outbox	Many internal consumers or multi-region
Database	PostgreSQL + Redis cache/locks	Measured scale need
Language	TypeScript/Node for services; official Stellar JS SDK	Measured bottleneck/team change
Soroban	Out of v1	Concrete programmable policy requires it
SEP path	SEP-6 target + SEP-24 hosted option; others as required	Client/partner interoperability requirement
Mobile	Out of v1	Consumer distribution becomes a validated channel
MCP/agents	Out of v1	Recon/authorization are mature and customer demand exists
Kubernetes	Out of v1	Platform/multi-region operational need
Countries	Nigeria only	Naira Rail meets reliability metrics

Decision	Canonical v1 choice	Revisit when
Assets	USDC on Stellar only	Approved business/legal need for another asset

42. Reference architecture sources

These current Stellar references were checked while freezing this specification. They support the implementation assumptions around Anchor Platform, SEPs, asset identity and RPC. Regulatory/legal design still requires Nigeria-specific professional advice and signed partner arrangements.

- Stellar Developers - Anchor Platform: supported SEP implementations and anchor architecture.
- Stellar Developers - SEP-6 Programmatic Deposits and Withdrawals.
- Stellar Developers - SEP-24 Hosted Deposits and Withdrawals.
- Stellar Developers - Anchors: traditional financial rails ↔ Stellar model.
- Stellar Developers - Assets: asset identity is code + issuer.
- Stellar Developers - Verify Trustlines.
- Stellar Developers - APIs Overview / Stellar RPC: RPC is recommended for real-time access and is not an application backend or historical index.
- Stellar Developers - Software Versions: verify current protocol/SDK versions again immediately before implementation/deployment.

Official documentation navigation:

developers.stellar.org/docs/platforms/anchor-platform
developers.stellar.org/docs/platforms/anchor-platform/sep-guide
developers.stellar.org/docs/learn/fundamentals/anchors
developers.stellar.org/docs/learn/fundamentals/stellar-data-structures/assets
developers.stellar.org/docs/data/apis/rpc

Implementation instruction. This document is the canonical Kora v1 build specification. If an older Kora document describes pan-African routing, mobile apps, MCP, multiple SDKs, eleven services or broader payment products as immediate requirements, this v1 specification overrides that scope. Those capabilities belong to the roadmap unless a formal decision-log amendment moves them into v1.